

Reuse, but Verify: Certified Maintenance of Table Cells Computed by Large Language Models under Prompt Edits

ARJUN LOHAN¹

¹University of Southern California, Los Angeles, CA 90089 USA (e-mail: lohan@usc.edu)

Corresponding author: Arjun Lohan (e-mail: lohan@usc.edu).

This work received no external funding.

ABSTRACT Modern tables let users define columns in natural language: a language model computes each cell from a versioned prompt. After a prompt edit, systems recompute every cell or reuse stale ones, with unbounded error. Semantic incremental view maintenance (sIVM) certifies which cached cells may be reused. It groups cells into strata by cached value, recomputes a growing sample of each, and bounds each stratum's flip rate (the share of its cells a recompute would change) with an exact finite-population bound. A stratum is reused when that bound clears the user's error budget; a strict mode bounds the error of the reused cells alone. An impossibility floor follows: with the prompt unedited, no stratum reliably certifies below its own self-disagreement rate. On two public corpora and one primary model (28,000 labeled cells), a formatting-only edit certifies reuse of 82.6% of cells on average at budget 0.1 and 88.1% at 0.2 (failure probability 0.1; 1,000 replications). On the full 89,184-row corpus at budget 0.2, 90 fresh calls certify reuse of 81,424 cells, in a live run that measured the served model's drift over 55 days. On the calibration grid, with independent draws, a certificate exceeding its budget is issued with probability at most 2.7% per stratum, against a nominal 5%. At least one edit certifies on 6 of 6 further models from four providers; which edits are benign varies by model. The system, labels, and five versioned prompt-edit pairs are released under a source-available noncommercial license.

INDEX TERMS Caching, confidence bounds, incremental view maintenance, large language models, materialized views, prompt engineering, query processing.

I. INTRODUCTION

Interactive artificial intelligence (AI) tables turn a natural-language prompt into a *materialized view*: a column in which a language model computes every cell from the prompt and that cell's row. Each cell is one draw from the model's output distribution (Section II gives the notation). The category now ships in commercial products. Google Sheets evaluates an `AI()` formula per row and offers AI columns in tables [1], Airtable computes AI fields per record [2], and Notion autofills database properties with a model [3]. Snowflake Cortex applies `AI_COMPLETE` and `AI_CLASSIFY` to table columns in Structured Query Language (SQL) [4]. Microsoft's Excel `COPILOT` function shipped as a preview and was withdrawn in September 2026, a measure of how quickly this surface is moving [5]. These views are edited constantly: users iterate on prompts the way they iterate on spreadsheet formulas. Every edit invalidates the cache by construction,

and today's systems invalidate it wholesale. An edit to an 89,184-row column recomputes all 89,184 cells, or the user silently keeps stale values with no bound on how wrong they are.

Recent semantic data systems bound the error of *one-shot* query execution against a reference model whose answers are taken as correct (cascades, semantic operators, approximate processing with guarantees). Verified semantic caches bound reuse under a *fixed* definition (Section VII names both lines). Neither answers the maintenance question an interactive table poses: *after this specific edit $\pi \rightarrow \pi'$, which cached cells are still valid, and with what guarantee?*

A. RUNNING EXAMPLE

One column from our evaluation recurs throughout. Over 89,184 Stack Overflow (SO) developer profiles, a Boolean AI column asks “*Is this person a data-platform specialist?*”, judged from each profile’s role, databases, platforms, and tools. Its owner edits the prompt to insert two paragraph breaks and lowercase one word, changing no content word (the *formatting-only* edit of Section V; Section VI lists every edit we study). Recomputing means computing all 89,184 cells again. Keeping every cached value silently carries the edit’s flip rate, the share of cells a recompute would change, into the table (6.35% on this pair). Semantic incremental view maintenance (sIVM) instead groups the cached cells by their cached value into two *strata*, cached-TRUE and cached-FALSE. It recomputes a small seeded sample from each and reuses a stratum only when a statistical bound on its flip rate clears the user’s error budget α . The procedure errs with probability at most δ . On this column, at $\alpha=0.2$ and $\delta=0.1$, 90 fresh calls certify reuse of 81,424 of the 89,184 cells. A live run did so 55 days after the column was built, against a model that had drifted in the meantime, and the draws stored when the column was built lead to the same decision (Section V-E). The cells it refuses are the cached-TRUE stratum, which flips at 37.1% under the same edit on the labeled evaluation rows. A single bound over all the cells would clear the budget and reuse those cells too (Section V-C).

B. EMPIRICAL MOTIVATION

First, large language model (LLM) cells are stochastic even at temperature zero [6], [7]. The same prompt on the same row disagrees with itself 5.0% of the time on our Boolean workload and 9.5% at default temperature (Section V). “Did the edit change this cell?” is therefore only meaningful relative to a noise floor, the rate at which the model disagrees with itself under the unedited prompt. Second, edit direction is not monotone. Under a strictly *scope-widening* edit, 45.7% of cached-TRUE cells flipped. Third, an edit’s surface class is not a safe predictor of its semantic impact. On the same column, a synonym paraphrase written to preserve meaning flipped 8.90% of cells where a formatting-only edit flipped 6.35%. On a candidate-profile column from a second corpus, a formatting-only edit flipped 12.30% against 12.45% for a deliberate criteria change, rates that are indistinguishable. The study’s five edits are author-written, one per class per column, so they show that class can fail to predict flip rate, not how often it does. The certifier below never consults the class, and Section V shows the same edit’s flip rate varying widely across models. Both comparisons hold the column fixed deliberately: on the two column types we study, the column’s own stability moves the flip rate more than the edit does.

Sound reuse therefore has to be *measured*, by sampling a few fresh cells, bounding the flip rate of the rest statistically, and reusing only what clears the user’s error budget.

C. CONTRIBUTIONS

- 1) **Problem formalization** (Section II): certified cross-version reuse for stochastic prompt-defined views. Two estimands (the population quantities a guarantee is about) are well-defined under decode pinning. One is the realized false-reuse rate over the cells a certified stratum presents, which we certify by default. The other is the same rate over the reused cells alone, which the strict mode certifies at a correspondingly lower threshold. An impossibility floor follows: with the prompt unedited, a budget α well below a large stratum’s self-flip rate is rarely certifiable, and Section II bounds how rarely.
- 2) **The sIVM procedure** (Section III): a stratification by cached value, frozen before sampling, and an adaptive sample on a doubling schedule of *looks* (points at which the sample is inspected). A futility rule gives up on a stratum whose sample already exceeds the budget. Certification is per stratum, with an exact finite-population confidence bound and two guarantee targets (presented cells; strict reuse set). Deciding at several looks is *peeking*; the validity theorem survives it by splitting the failure probability δ across the looks. Any frozen stratifier affects power only, never validity, where power is the chance of certifying a stratum that does meet the budget. The exact bound is proven for the without-replacement sampling the procedure performs, and four alternatives (two empirical-Bernstein bounds, the binomial Clopper–Pearson bound, and a betting confidence sequence) are measured inside the same procedure.
- 3) **A working implementation** (Section IV) inside an AI-table system, released under a source-available non-commercial license. The system has a per-cell runner, a content-hash cache, a cost ledger, and certificates (the persisted records of what cleared the budget), which the user interface (UI) surfaces.
- 4) **An empirical anatomy** (Section V) of one primary model on two real corpora with 28,000 labeled cells. It measures the noise floor, the savings frontier over the error budget, and minority-stratum volatility, with how much of it majority voting at write time removes. It includes a stratifier ablation and a calibration study of how often the procedure issues a certificate that exceeds its budget. It checks the cross-row independence assumption. It also covers 6 further models from four providers. A re-certification 55 days after the column was built measured drift of the served model as a higher flip rate against the same cache, and certified both budgets again. A free-text pilot (Section VIII) calibrates a judge’s error on hand labels and folds it into the budget.
- 5) **A reusable benchmark** (Section VI): five versioned prompt-edit pairs with per-cell flip labels over public corpora redistributable under the Open Database License (ODbL) and the MIT License, so that future

maintenance systems can be evaluated like for like.

The remainder of this article is organized as follows. Section II formalizes certified reuse and its two guarantee targets. Section III presents the sIVM procedure and its validity theorem. Section IV describes the system it runs in. Section V reports the evaluation. Section VI describes the released benchmark of versioned prompt-edit pairs. Section VII positions the work against prior art. Section VIII discusses which budgets are usable and states the limitations, and Section IX concludes. The Supplementary Material holds Appendices A to J, with the detail of each study and the tables and figures numbered S1, S2, and so on.

II. PROBLEM

a: Terminology.

Table 1 defines the terms and the symbols this article uses. Four terms carry most of the weight. A *stratum* is a group of cached cells fixed before any sampling (by default, the cells that share a cached value). A *look* is a point in the sampling schedule at which the sample drawn so far is inspected and a decision may be taken. A stratum's *floor* is its self-flip rate, the rate at which the model disagrees with its own earlier answer when nothing has changed. A *certificate* is the persisted record that a stratum cleared the budget, with the evidence it cleared it on.

b: Views and cells.

An AI column is a view $V = (\pi, \mathcal{M}, \theta)$: a prompt template π with a typed output contract, a model $\mathcal{M}$, and pinned decode parameters θ . For row r_i , the instantiated prompt $\pi(r_i)$ interpolates only the fields $\beta(\pi)$ that π references; the stored cell c_i is *one draw* from the output distribution $f_V(r_i)$. Even at temperature zero this distribution is not degenerate: providers are nondeterministic across calls, as prior measurements of hosted models document [6], [7], in part because the floating-point reduction order inside serving kernels changes with the co-batched load [8], [9]. Every quantity below is therefore defined over draw randomness.

c: The maintenance event.

The user edits the definition, $V \rightarrow V' = (\pi', \mathcal{M}, \theta)$. For each cached cell define the flip indicator $F_i = \mathbf{1}[c_i \neq X'_i]$, where $X'_i \sim f_{V'}(r_i)$ is a fresh draw under the new definition and $\equiv$ is the column's equivalence relation (exact match for the Boolean and select columns studied here). The indicator F_i is Bernoulli with unknown mean $p_i = \Pr[c_i \neq X'_i]$. That mean folds together the edit's semantic effect and decode noise on both sides: it answers the operational question of whether recomputing cell i now would return a value different from the one shown.

d: Estimand and guarantee.

A maintenance policy partitions the cached cells into a sampled set $\mathcal{S}$ (freshly recomputed), a reuse set $\mathcal{R}$ (stale values kept), and a recompute set. Two error rates matter, and they differ. The *presented-cells* rate of a certified stratum j is

$$\text{FR}_j^{\text{pres}} = \frac{1}{n_j} \sum_{i \in j \setminus \mathcal{S}} F_i :$$

of all n_j cells the user now sees from that stratum, the fraction that would flip if recomputed (sampled cells are fresh and contribute zero). That zero is a convention of the single-draw estimand. A recomputed cell is itself one draw, and a further draw would disagree with it at about the stratum's self-flip rate. The estimand charges reuse for exactly what the user would see change on recomputation, which is the operational question. The impossibility floor below is the price of that choice: an excess-over-floor target, $p_j - s_j$, would certify the identity edit for free. We deliberately do not adopt that target, because a user who recomputes sees one draw and lives with it. The *reuse-set* rate

$$\text{FR}_j^{\text{reuse}} = \frac{1}{n_j - m_j} \sum_{i \in j \setminus \mathcal{S}} F_i$$

normalizes by the $n_j - m_j$ reused cells alone and is never smaller. A certificate bounds the stratum's whole flip count, and hence $\text{FR}_j^{\text{pres}}$, but not $\text{FR}_j^{\text{reuse}}$: the remainder is adversely selected, because certification happens exactly when the sample looked clean. sIVM therefore certifies the presented-cells estimand by default, and offers a strict reuse-set mode that certifies at the lower (*deflated*) threshold $\alpha(n_j - m_j)/n_j$, which bounds $\text{FR}_j^{\text{reuse}}$ by conservation of flips. On the *headline pair*, the SO formatting-only edit of the running example, at $\alpha=0.2$ the two rates read 3.20% and 3.28% (Table 2). Theorem 1 states both guarantees for the realized rates of this run, not for their expectations. All guarantees are per-stratum and therefore hold simultaneously for every union of certified strata, not only the aggregate (baseline B2 in Section V-C, aggregate-only certification, shows why the aggregate alone is operationally misleading).

sIVM makes no decision about an individual cell. Whether one cached cell would change under the edited prompt can be learned only by recomputing it, which spends exactly the call that reuse would save. The cheapest per-cell signal, prompt similarity, predicts flips at chance across a definition change (baseline B1 in Section V-C). The signals that do carry information, the cached value and how strongly the edit's changed terms match the row (the interaction score of Section III), separate groups of cells, not single cells, and enter as strata. The unit of decision is therefore the stratum. An unsampled cached cell is reused if and only if the upper confidence bound on its stratum's flip rate clears the budget at a look the schedule reaches. That is the correctness criterion for reuse, and it is a statement about recomputation, not about ground truth. A certificate asserts that recomputing the stratum now would change at most an α share of its cells. It does not assert that the cells are right.

TABLE 1. Terms and notation.

Term	Meaning
stratum	a group of cached cells fixed before any sampling; by default the cells that share a cached value
look	a point in the sampling schedule at which the sample so far is inspected and a decision may be taken; named by its sample size (look 45, look 90)
flip	a fresh draw under the edited prompt that disagrees with the cached value
floor	a stratum's self-flip rate: how often a fresh draw under the <i>unedited</i> prompt disagrees with the cached value
certificate	the persisted record that a stratum cleared the budget, with its evidence
refused	said of a stratum that is not certified; its unsampled cells are recomputed
futility	stopping a stratum's sampling because the sample's own flip rate already exceeds the budget
oracle draw (call)	one fresh model call under the edited prompt for one row
label	a stored fresh draw, used as the outcome a recompute would give; a model draw, not a human judgment
savings	the share of a column's cells kept under a certificate (neither sampled nor recomputed)
frontier	savings as a function of the budget
loose, tight	$\alpha=0.2$ and $\alpha=0.1$
budget	
estimand	the population quantity a guarantee is about
power	the chance that a stratum which meets the budget is certified
null	a stratum whose realized flip count exceeds the budget; a <i>planted</i> null is a simulated stratum generated at a flip rate above the budget
unsafe	a certificate issued for a null
certificate	

Symbol	Meaning
$V = (\pi, \mathcal{M}, \theta)$	a view: prompt template, model, pinned decode parameters; $V' = (\pi', \mathcal{M}, \theta)$ after the edit
r_i, c_i	row i and its cached cell, one draw from $f_V(r_i)$
X'_i, F_i, p_i	a fresh draw under V' ; the flip indicator $\mathbf{1}[c_i \neq X'_i]$; its mean
$\equiv$	the column's equivalence relation (exact match here); the theorem needs only the binary indicator it yields
j, n_j, s_j, p_j	a stratum, its size, its self-flip rate (floor), its flip rate (the mean of p_i over $i \in j$)
$\Delta, \beta(\pi)$	the prompt delta (content terms added or removed); the row fields π interpolates
$\mathcal{S}, m_j$	the sampled (freshly recomputed) cells; the number of them in stratum j
$\mathcal{R}$	the reuse set: cached cells kept under a certificate
L_j, t, n_t	stratum j 's look schedule, a look in it, and the cells drawn by that look; n_0 is the first look and ℓ the most looks allowed
N, n, k	in the bound: the stratum's size ($N = n_j$), the cells drawn at a look, and the flips among them
$u_{j,t}, \hat{p}_j$	the upper confidence bound on stratum j 's flip rate at look t ; the sample's own flip rate
α, α', δ	error budget on the false-reuse rate; the strict mode's deflated threshold; upper bound on the probability that any certificate is wrong
K, δ_j, δ'	number of strata; the per-stratum failure probability δ/K ; the per-look level $\delta_j/ L_j $
M_j, k_j	stratum j 's realized flip count; the flips observed in its sample
$\text{FR}_j^{\text{pres}}$	realized false-reuse of stratum j over the cells it presents
$\text{FR}_j^{\text{reuse}}$	the same over its reused cells alone

Term (continued)	Meaning
replication	a rerun of the certifier on the same labels with another sampling seed; the <i>main seed</i> gives the one run reported per configuration
snapshot	the model as served under one identifier over one period; a change of the served model ends it
pinned	held fixed: the decode parameters during a certification, and the one configuration every experiment uses
Bonferroni split	dividing a failure probability among several tests so that their failure probabilities cannot add up past it

e: Reading the budget.

The guarantee is a statement about the procedure, over a run's sampling and oracle draws. With probability at least $1 - \delta$ the run issues no wrong certificate: none for a stratum in which more than an α fraction of the cells (in the strict mode, of the reused cells) would flip if recomputed. The guarantee is not the probability that a certificate in hand is right. A stratum whose flip rate sits just above the budget is certified rarely, and every certificate it does receive is wrong; Table 2 contains such a configuration (Section V-B). Both α and δ are the user's to set. A list that a person keeps working through tolerates a looser α than a column that gates a decision (Section VIII). The realized error under a certificate is typically far below α (3.20% against $\alpha=0.2$ on the headline pair, Table 2), because the bound must cover the worst population compatible with the sample.

f: Assumptions.

Assumption 1 (Pinned snapshot). The model $\mathcal{M}$ and the decode parameters θ are fixed during certification. Oracle draws are distributed as $f_{V'}(r_i)$ throughout the certification window and are independent *across rows*. Provider-side batching that correlates concurrent responses [8], [9] would violate this; our runner issues independent requests, and Section V-F compares sequential with concurrent requests. The validity proof uses independence only through non-interference: a sampled cell's draw must not depend on which other cells were sampled or in flight. Rows with identical bound content share one cell by design (Section IV), and hence one draw; that sharing is fixed by row content before sampling and leaves non-interference intact. Certificates are scoped to the snapshot the oracle draws came from; the cache may be older. A change of the served model ends the snapshot, and Section V-E (the

September run) measures such a change (change-of-world maintenance is complementary work).

Assumption 2 (Frozen stratification). The stratifier is a deterministic function of the cache, the edit, and row content, fixed before any oracle call. Any such stratifier preserves validity; its quality affects only power (Section V-G).

Assumption 3 (Uniform within-stratum sampling). Sampled rows are drawn by a seeded uniform shuffle within each stratum, without replacement, and each sampled row receives one fresh oracle draw, independent across rows (Assumption 1, which also covers rows that share a cell). The sampling itself is fixed by the seeded canonical shuffle of Section IV. The calibration study of Section V-F checks that the implemented test holds the level Theorem 1 states, on planted strata whose rates sit just above the budget and on the fixed population that is least favorable to it.

g: No monotonicity shortcuts.

A scope-widening edit need not preserve TRUE cells. Under our widening edit 45.7% of cached-TRUE cells flipped, although the widened definition admits every row the old one did. Edit direction therefore enters sIVM only as a stratification hint; any directional certificate must be *conditional* on a measured violation rate, never assumed.

h: The impossibility floor is per-stratum.

Let s_j be stratum j 's self-flip rate: the probability that a fresh draw of a cell under the *unedited* prompt disagrees with the cached draw, among the cells whose cached draw put them in stratum j . For the identity edit $\pi' = \pi$, p_j equals s_j by definition. The floor concerns certificates that bound a stratum's realized flip count M_j , the count that charges reuse for every disagreement with one fresh draw. Both modes of sIVM issue such certificates. Take any procedure that certifies a stratum with $M_j > \alpha n_j$ with probability at most δ_j . For the identity edit at a budget $\alpha < s_j$ it certifies stratum j with probability at most

$$\delta_j + \Pr[M_j \leq \alpha n_j] \leq \delta_j + \exp(-2n_j(s_j - \alpha)^2),$$

where the second step is Hoeffding's inequality [10] for draws that are independent across cells. (If w_c rows share cell c , the exponent's n_j becomes $n_j^2 / \sum_c w_c^2$.) Under the unedited prompt the certifiable budget is therefore floored by model stability, up to a term that vanishes as the stratum grows. For a budget just below the floor that term is not small, and there power, not the floor, is what limits certification. Three scoping notes apply. First, the floor limits bounds on the whole count. A rule that bounds only the presented-cells rate can go below it by sampling more, because sampled cells are recomputed; in the limit it samples the whole stratum and reuses nothing. Second, because membership is decided by one draw, a minority-value stratum is selected partly for having flipped that way, so s_j is a property of the (cache, stratifier) pair. Building the cache by a majority vote over several draws at write time lowers it (Section V-G). Third, the

floor is a bound for the identity edit only. An edit that sharpens the output distribution, or shifts it toward a stratum's cached value, can drive p_j below s_j , and the model study contains such strata (Section V-D). For every other edit the floor is a reference point, not a bound.

The floors are unequal, and the column-level average describes neither stratum. We measure each floor as defined, the stored cached value against three fresh draws of the same prompt per row. On our Boolean column the cached-FALSE stratum has $s=2.6\%$ (95% row-cluster bootstrap [2.0, 3.1]) while the cached-TRUE stratum has $s=23.0\%$ ([18.6, 27.4]), against 4.5% pooled. The column-level 5.0% quoted elsewhere is the rate between pairs of fresh draws. First, the FALSE stratum is the only one that certifies at all, and it is certifiable in principle down to budgets near its own floor. At $\alpha=0.05$ it certifies in 74.4% of replications, some only at the deepest look of the schedule, for 38.6% mean savings (Table 2). There, power rather than the floor is what binds. Second, the TRUE stratum is never certified at any budget we tried: its estimated floor is above the loosest of them, though the interval reaches below it. Its size sets the corpus-scale savings ceiling: with 7,715 of 89,184 cells cached TRUE, reuse from this cache cannot exceed 91.3%. Third, any method that reuses cells from the cached-TRUE stratum inherits that stratum's flip rate no matter what aggregate target it was given, which is what we observe when we run one (Section V-C, B2'). Stratifying by cached value isolates that subgroup, which a column-level rate conceals.

III. THE sIVM PROCEDURE

Given the edit $\pi \rightarrow \pi'$ and cached cells for the affected rows (Fig. 1):

- 1) **Diff.** Compute a word-level longest-common-subsequence (LCS) diff of the prompts; collect content terms of added/removed spans (the *delta*). The delta is recorded in the certificate for provenance and consumed only by the optional refinement in step 2; it never enters the reuse decision itself.
- 2) **Stratify (frozen).** Partition rows by cached value and freeze the partition before any oracle call (Assumption 2). With a binary proxy this step is the stratification of proxy-certified selection [11], transported from one-shot selection to maintenance. What changes is the guarantee it carries: a bound per stratum instead of one aggregate bound over the selected set (Section V-C, B2', isolates exactly that difference). Optionally refine each value stratum by an *interaction score*, a measure of how strongly the delta matches the row (lexical delta-term overlap, or the embedding cosine between the delta text and the row's bound content), bucketed into tertiles. This refinement leaves validity intact (Assumption 2). It pays only when within-value flip risk is heterogeneous, and otherwise costs power through the δ/K split (Section V-G).
- 3) **Schedule.** For stratum j of size n_j , fix a doubling look schedule $L_j = (45, 90, 180, \dots)$ capped at n_j , with at

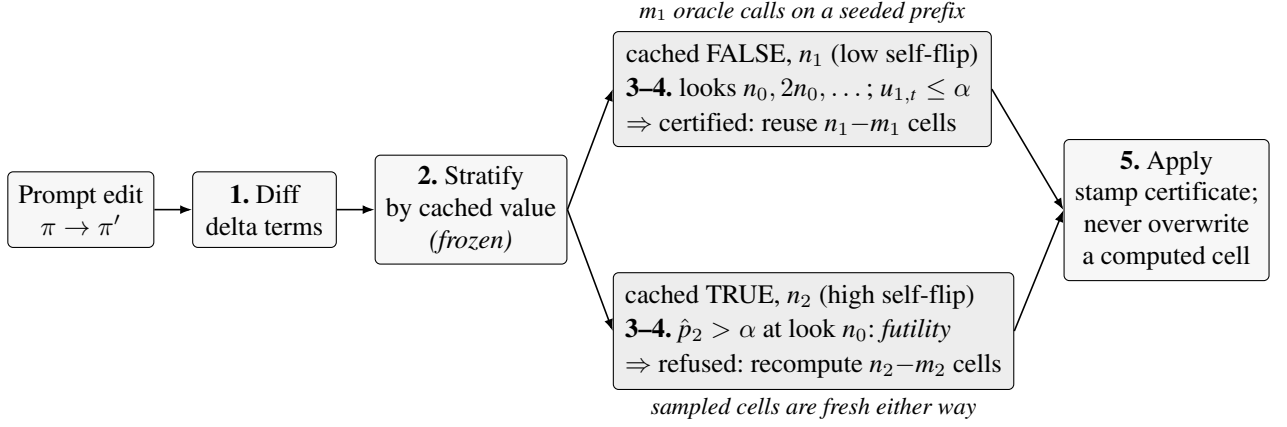


FIGURE 1. The procedure on the configuration our experiments pin. Stratification is frozen before any oracle call, so it moves power but never validity (Assumption 2); only steps 3–4 spend model calls, and they stop on evidence about a stratum’s *rate*, so their cost is bounded independently of the stratum’s size. The two lanes are the outcome on the primary model’s Boolean column: the stable stratum certifies and is reused under a certificate, while the volatile one is refused at every budget we tried and recomputed. A refusal is a stability diagnosis of that slice of the column (Section II), reached at the cost of one look instead of a full recomputation.

most six looks. Split the stratum’s failure probability $\delta_j = \delta/K$ evenly across looks. Every experiment here pins $n_0=45$, at most six looks, and $\delta=0.1$. Here K is the number of cached values: 2 on the Boolean columns and 4 on the select column. A look on a large Boolean stratum is tested at the per-look level $\delta' = \delta_j/|L_j| = 0.1/12$, and a look on a large select stratum at $0.1/24$. A small stratum has fewer looks and a correspondingly larger per-look level.

- 4) **Sample and test.** Order the stratum’s rows canonically, shuffle them once with the seed (Assumption 3), and at each look t reveal one fresh draw for each row of the prefix (Assumption 1). Compute the exact finite-population upper confidence bound $u_{j,t}$ of Eq. (1) on the stratum flip rate at level $\delta_j/|L_j|$; *certify and stop* if $u_{j,t} \leq \alpha$; *stop for futility* if the empirical rate alone exceeds α .
- 5) **Apply.** Reuse all unsampled rows of certified strata (recording certificate provenance per cell); recompute the rest. Sampled rows are already fresh.

Algorithm 1 states the procedure in full. What the test conditions on is fixed before the first oracle call: the stratification, the look schedule with its per-look levels, the seed, the bound, and the equivalence relation. The budget α and the mode enter where a look’s bound is compared with a threshold, and in the futility stop, which can only end sampling; the schedule’s last look is sized from α only for budgets near 0.01 or below. The event on which every scheduled look’s bound holds therefore does not depend on them, so several budgets, and both modes, can be read from one sample path at no cost in δ . Two results were computed after their draws had been seen, and are marked as retrospective where they appear: the replay of the August draws under the exact bound (Section V-E) and the graded relation of Section V-B. The guarantee is per certification: over m certifications the chance that at least one certificate is wrong is at most $m\delta$.

Algorithm 1 sIVM certification of an edit $\pi \rightarrow \pi'$

Require: cached cells $\{(r_i, c_i)\}$; edit $\pi \rightarrow \pi'$; budget α ; failure probability δ ; first look n_0 ; at most ℓ looks; mode $\in \{\text{default}, \text{strict}\}$; Assumptions 1–3

Ensure: reuse set $\mathcal{R}$, recompute set, certificate record

- 1: $\Delta \leftarrow$ word-level LCS diff of π and π'
 - 2: strata $1, \dots, K \leftarrow$ partition of the cells by cached value, optionally refined by an interaction bucket of Δ ; freeze
 - 3: **for** each stratum j with n_j cells **do**
 - 4: $\text{order}_j \leftarrow$ seeded shuffle of the stratum’s rows in canonical order; $L_j \leftarrow (n_0, 2n_0, \dots)$ capped at n_j , at most ℓ looks; $\delta' \leftarrow \delta/(K |L_j|)$
 - 5: **for** each look $n \in L_j$ **do**
 - 6: draw X'_i for the first n rows of order_j not yet drawn; $k \leftarrow |\{i \leq n : c_i \neq X'_i\}|$
 - 7: $u \leftarrow$ exact upper bound of Eq. (1) at (k, n, n_j, δ')
 - 8: $\alpha' \leftarrow \alpha$ in the default mode, else $\alpha(n_j - n)/n_j$
 - 9: **if** $n < n_j$ **and** $u \leq \alpha'$ **then**
 - 10: certify stratum j ; **break**
 - 11: **end if**
 - 12: **if** $k/n > \alpha$ **then**
 - 13: refuse stratum j (futility); **break**
 - 14: **end if**
 - 15: **end for**
 - 16: refuse stratum j if its schedule is exhausted
 - 17: **end for**
 - 18: $\mathcal{R} \leftarrow$ unsampled cells of certified strata; recompute set $\leftarrow$ unsampled cells of refused strata
 - 19: persist the certificate (budget, Δ , per-stratum evidence) and apply $\mathcal{R}$ additively
-

α : The bound.

Fix a stratum of N cells ($N = n_j$), and give every cell one hypothetical fresh draw under V' , the coupling on which the presented-cells estimand is defined. Let M be the number of

cells whose fresh draw flips. The sample of n cells is drawn uniformly without replacement and independently of those draws, and shows k flips. Conditional on the N outcomes, k is Hypergeometric(N, M, n), and $\Pr[\text{Hyp}(N, M, n) \leq k]$ is non-increasing in M . The exact upper confidence bound at level δ' is

$$u = \frac{1}{N} \max\{M : \Pr[\text{Hyp}(N, M, n) \leq k] > \delta'\}, \quad (1)$$

the finite-population form of the Clopper–Pearson bound [12], to which it tends as N grows. Because the coverage holds conditionally on every population realization, it holds unconditionally over sampling and draw randomness together, and it bounds the realized flip count M rather than its expectation. This is the natural bound for a binary loss under the sampling the procedure performs. On a clean sample its cost is $\ln(1/\delta')/n$ to first order. The empirical-Bernstein bounds pay range terms with constants of $7/3$ (Maurer–Pontil, on $\ln(2/\delta')$) and $7/3 + 3/\sqrt{2}$ (Bardenet–Maillard, on $\ln(5/\delta')$) that a binary loss does not need. On a clean sample of $n=90$ from a stratum of 1,800 cells at the pinned per-look level, the exact bound reads 0.050, the Maurer–Pontil bound [13] 0.144, and the Bardenet–Maillard without-replacement bound [14] 0.317. Section V-G measures the difference on every configuration, and the system exposes the alternatives as ablation arms.

Theorem 1 (Validity under peeking). Let K be the number of strata produced by the frozen stratifier, $\delta_j = \delta/K$, and let every look use the exact bound of Eq. (1) at level $\delta_j/|L_j|$. Under Assumptions 1–3, with probability at least $1 - \delta$ over sampling and oracle draws, every certified stratum j has realized whole-stratum flip count $M_j \leq \alpha n_j$; consequently $\text{FR}_j^{\text{pres}} \leq \alpha$ for every certified stratum and for every union of certified strata (the empty certification is vacuously valid). In the strict (reuse-set) mode, where certification at sample size m_j requires $u_{j,t} \leq \alpha(n_j - m_j)/n_j$, the same argument gives $M_j \leq \alpha(n_j - m_j)$, hence $\text{FR}_j^{\text{reuse}} \leq \alpha$. The statements hold for any per-look bound that is a valid upper confidence bound at that level for uniform without-replacement draws, including the Bardenet–Maillard bound [14]; the Maurer–Pontil bound [13] is stated for independent draws and is retained only as an ablation arm.

Proof. Fix stratum j and condition on its n_j fresh-draw outcomes, of which M_j flip. If $M_j > \alpha n_j$, certifying at look t requires $u_{j,t} \leq \alpha < M_j/n_j$, i.e., the observed count to fall in the lower tail that Eq. (1) rejects at level $\delta_j/|L_j|$; by the monotonicity of the hypergeometric tail in M , that event has probability at most $\delta_j/|L_j|$ over the sampling, for every population realization, hence unconditionally. Because the looks are prefixes of one shuffle fixed before sampling, the prefix of size n_t (the t -th entry of L_j) is, marginally, a uniform without-replacement sample of that size, whether or not the procedure reaches look t . The bound therefore fails at any one scheduled look with probability at most $\delta_j/|L_j|$, and a union over all scheduled looks covers whichever look the data stops

at. That union bound over the $|L_j|$ looks and the K strata gives δ ; this is the Bonferroni instance of Learn then Test [15], each (stratum, look) pair testing the null $M_j > \alpha n_j$ with a certificate as the rejection. The futility rule only stops sampling, never certifies, so it cannot increase the error probability. On the good event, $\text{FR}_j^{\text{pres}} = (M_j - k_j)/n_j \leq \alpha$, where k_j is the number of flips observed in the sample, and any size-weighted union of certified strata inherits the bound. In the strict mode the good event is $M_j \leq \alpha(n_j - m_j)$; at worst every flip falls in the remainder, so $\text{FR}_j^{\text{reuse}} \leq \alpha$. The conditioning uses Assumption 1 only through non-interference: the population of fresh-draw outcomes must be the same whichever cells the shuffle selects. $\square$

Remark. The Bardenet–Maillard bound is proven for the same uniform without-replacement draws (its constants are those of Theorem 4.3 of [14]), so the argument transfers to it unchanged. The Maurer–Pontil bound is stated for independent draws, and we do not prove it valid at the sampled fractions our runs reach, which is why the pinned bound is the exact one. Those fractions are large: the schedule’s deepest look draws 1,440 cells of a stratum, and the certifier samples up to 74% of a benchmark table.

b: Power.

In a large stratum a clean sample certifies once $(1 - \alpha)^n \leq \delta'$, i.e., at $n^* = \lceil \ln \delta' / \ln(1 - \alpha) \rceil$. Under the pinned procedure on a Boolean column ($K=2$, six looks), this gives $n^*=94$ at $\alpha=0.05$, against 257 for the Maurer–Pontil bound. The doubling schedule cannot stop on n^* : the first look at or above it is the one it takes (Appendix F gives the draws a clean stratum pays in calibration). Savings therefore require strata much larger than n^* , i.e., large tables, the deployment regime. Near the frontier, certifying a stratum with true rate p requires roughly $n \gtrsim z_{\delta'}^2 p(1-p)/(\alpha-p)^2$ samples, where $z_{\delta'}$ is the standard normal quantile at level δ' (the normal approximation to the exact test). This explains the measured collapse of savings as $\alpha \downarrow p$. On SO formatting (Table 2), mean savings are 88.1%, 82.6%, and 38.6% at $\alpha=0.2$, 0.1, and 0.05, with refusal at 0.02.

IV. IMPLEMENTATION

sIVM runs inside *lore*, a working AI-table system. Its natural-language search bar compiles to a versioned filter specification executed identically on MySQL and Elasticsearch (conformance-tested for count, page, sort, aggregation, and hydration parity on 89,184 rows). AI columns are per-cell jobs with typed structured outputs, bounded concurrency, retry, and a content-hash cache keyed on *only the referenced fields* $\beta(\pi)$. Unrelated row changes therefore never invalidate the cache, and content-duplicate rows share one computation. Every run passes a pre-run cost gate and lands in a ledger; prompt edits create immutable versions (the sIVM substrate); certificates persist with full strata evidence, and reused cells carry visible provenance in the UI. Decode is pinned ($T=0$) for all cell computation per Assumption 1. The table’s certify

action runs the pinned procedure of Section III in the default mode on the rows of the user's current view, at most 2,000 per request. Each request is a certification of its own, with its own δ . The corpus-scale runs of Section V-E call the same certifier function from a script.

a: What the user sees.

In the table view a reused cell carries a shield mark whose tooltip shows the cell's stored rationale and notes that it was reused under a statistical certificate. Certifying a column version returns the certificate's summary: the rows in scope, α , the cells sampled, reused, and to be recomputed, and the flips observed. A refused stratum's cells are recomputed and served as ordinary fresh cells; the refusal is visible in the certificate record. Whether this surface changes how a person acts on the column was not studied (Section VIII).

b: Storage model.

Cells are keyed by (column, row, prompt version), so versions of a column *coexist* instead of replacing one another, and a secondary index on (column, version, model, content hash) serves the cache. This is the one design decision sIVM depends on. A system that keys a cell by row alone, or hashes the prompt into the cache key, destroys the old materialization at the moment of the edit, so nothing is left to certify. Keeping versions side by side turns "which cached cells still transfer?" into a join between two versions of one table, so the certifier is a query over the store, not a service beside it. It orders each stratum's rows canonically (by content hash, then row identifier) before the seeded shuffle, so the sample path is a function of the seed and the data alone. The certifier used for the August run took the order the storage engine returned, which changed when the query planner switched indexes, so the same seed drew a different sample (Appendix C). Oracle draws taken after a change of the served model are stored under a new prompt version carrying the same template, so draws from two snapshots never share a key and cannot be mixed in one certification path.

c: Certificates are records, not decisions.

A certificate persists the budget α , the prompt delta, the per-stratum evidence (size, cells sampled, the bound at each look, certified or refused), and the counts of sampled, reused, and recomputed cells. Each stratum's entry also names the level, the mode, and the bound it was tested under; that field is new in this release, and the certificates of Section V-E carry those settings in their run artifacts. A certificate is therefore auditable after the fact: the realized-error audits in Section V-E are recomputed, from the row identifiers each run persisted and the stored cells, by a script that imports nothing from the certifier. Applying a certificate is a single `INSERT . . . SELECT` that carries certified values forward under the new version, stamps the certificate identifier on each one, and anti-joins against the target version. Reuse is therefore strictly *additive*: a certificate can supply a value where none was computed, and can never overwrite one that

was. An earlier implementation used an upsert, the obvious way to write it, and applying a certificate after a partial recompute silently reverted freshly computed cells to stale values. The bug is invisible in aggregate error, because a reverted cell and a reused cell carry the same value, so the guard belongs in the write path, not in code review.

Three further rules govern live operation. A sampled cell whose oracle call fails is counted as a flip and left for recomputation. The rule is conservative: a failure can add to the flip count the bound sees and never subtract from it, so it can cost a certificate and cannot create one. (The studies of Section V measure flip rates on completed draws; Appendix D reports the independence check under both conventions.) Reuse does not chain. A reused cell is a copy of the previous version's value, not a draw of the new one, and the certifier takes only computed cells as its cache. At the next edit a reused cell is therefore recomputed, and the savings reported here are per edit (Section VIII). Finally, the sampling seed defaults to a hash of the column and its two versions. Successive certifications of one column therefore draw different samples, and each sample is reproducible from the certificate record. The experiments of this article set the seed explicitly (Section V-A).

V. EXPERIMENTS

This section reports the setup (Section V-A), the main results (Section V-B), the baselines (Section V-C), the other models (Section V-D), the deployment-scale runs (Section V-E), the validity checks (Section V-F), and the ablations (Section V-G).

A. SETUP

a: Workloads.

All measurements use two public corpora and three prompt-defined columns (Section VI gives the exact edits, and the Availability section the sources). On the 2023 SO Developer Survey (89,184 respondent rows) a Boolean column asks whether the respondent is a data-platform specialist, judged from structured facets of the row (role, databases, platforms, tools). The column exists twice, as the production column of the system's own table and as a lab replica seeded from that column's second version, so that edits can be studied without touching the first. (The system has no outside users; "production" names the column.) On Djinni English candidate profiles (210,250 rows) a four-way ordinal select column assigns a seniority tier from the position, years of experience, and free-text profile. Five edit pairs across the three columns cover formatting-only edits, synonym rewording, scope widening, and a criteria change; each is labeled on a seeded $n=2,000$ evaluation vector of its corpus.

TABLE 2. The pinned primary procedure (cached-value strata; exact bound; adaptive doubling looks $n_0=45$, six looks; $\delta=0.1$; presented-cells estimand) on all five edit pairs, $n=2,000$ each at temperature 0. Column *flip* is the pair's label flip rate. Columns *oracle* and *reused* are the calls spent and the cells reused by the main seed. FR_{pres} and FR_{reuse} are the main seed's realized false-reuse over presented cells and over the reuse set alone. Column *savings* is the mean over $B=1,000$ sampling replications on frozen labels, with a 95% percentile interval (PI). Column *cert.* is the share of those replications that issued any certificate. Column *exceed.* is the share of the certifying replications (not of B) whose realized reuse-set rate exceeded α , a metric the default mode does not certify (Section II). The interval is taken over the replications, so a configuration that certifies in only a few of them shows a mean above an interval of zeros (scope widening at $\alpha=0.05$). The table is generated mechanically from the result files.

Edit	α	flip	oracle	reused	FR_{pres}	FR_{reuse}	savings % [95% PI]	cert.	exceed.
SO formatting-only	0.02	6.35%	225	0	—	—	0.0 [0.0,0.0]	0.0%	—
SO formatting-only	0.05	6.35%	1,485	374	0.55%	2.67%	38.6 [0.0,81.7]	74.4%	0.0%
SO formatting-only	0.10	6.35%	90	1,769	3.20%	3.28%	82.6 [72.7,88.5]	98.9%	0.0%
SO formatting-only	0.20	6.35%	90	1,769	3.20%	3.28%	88.1 [86.2,88.5]	100.0%	0.0%
SO synonym rewording	0.02	8.90%	90	0	—	—	0.0 [0.0,0.0]	0.0%	—
SO synonym rewording	0.05	8.90%	90	0	—	—	0.0 [0.0,0.0]	0.0%	—
SO synonym rewording	0.10	8.90%	90	0	—	—	55.6 [0.0,89.0]	81.4%	0.0%
SO synonym rewording	0.20	8.90%	225	1,645	5.81%	6.44%	87.4 [82.3,89.0]	99.9%	0.0%
SO scope widening	0.02	9.45%	90	0	—	—	0.0 [0.0,0.0]	0.0%	—
SO scope widening	0.05	9.45%	765	0	—	—	1.0 [0.0,0.0]	1.2%	100.0%
SO scope widening	0.10	9.45%	405	1,441	4.50%	5.62%	66.3 [0.0,87.8]	88.8%	0.0%
SO scope widening	0.20	9.45%	90	1,756	5.33%	5.47%	86.6 [85.6,87.8]	99.9%	0.0%
Djinni formatting-only	0.02	12.30%	180	0	—	—	0.0 [0.0,0.0]	0.0%	—
Djinni formatting-only	0.05	12.30%	315	0	—	—	0.0 [0.0,0.0]	0.0%	—
Djinni formatting-only	0.10	12.30%	495	383	4.17%	8.09%	8.4 [0.0,32.7]	73.8%	8.4%
Djinni formatting-only	0.20	12.30%	953	1,047	8.33%	10.12%	52.1 [32.7,73.6]	100.0%	0.0%
Djinni criteria change	0.02	12.45%	180	0	—	—	0.0 [0.0,0.0]	0.0%	—
Djinni criteria change	0.05	12.45%	315	0	—	—	0.0 [0.0,0.0]	0.0%	—
Djinni criteria change	0.10	12.45%	540	378	3.52%	6.88%	14.4 [0.0,32.4]	82.8%	5.4%
Djinni criteria change	0.20	12.45%	745	1,255	7.50%	11.00%	48.6 [32.4,69.5]	100.0%	0.0%

b: Scope.

One low-cost reasoning model carries the full experimental program. A further 6 models from four other providers, three of them successive Gemini generations, replicate the lab column's two pairs; 5 of them replicate all five pairs. The quantitative frontier reported below is specific to the (edit, model) pair, while the qualitative structure is what the model study tests (Section VIII). Table S9 lists every model identifier and run date. The gateway exposes no snapshot identifier finer than these names (only the primary model's carries a date suffix), so the run dates are the snapshot record, and what one identifier serves can change between them (Section V-E).

c: Models and decoding.

Every cell in the primary program uses DeepSeek V4 Flash through the Vercel AI Gateway (identifier `deepseek/deepseek-v4-flash-0731`). Those calls were made between 4 August 2026 and 5 August 2026, and again on 29 September 2026 for the re-certification and the assumption checks (and on 30 September 2026 for the larger concurrent arm of the independence check). The model study calls the further models through the same gateway from 15 August 2026 to 29 September 2026. Every call passes temperature 0 and no other decode parameter; maximum output length and reasoning effort stay at the provider's defaults, and no decode seed is sent. The seeds of this article govern row sampling only: one main seed gives the single run reported per configuration, and 1,000 further seeds give the sampling replications. Cells are requested as typed structured outputs against a JavaScript Object Notation (JSON) schema, and the client retries a call whose output

fails the schema or whose request errors. A cell that still fails is stored with an error status and excluded from labels and from every rate; none exists among the cells written through the column runner, and the probe scripts that call the model directly report their own exclusions. The embedding proxy in baselines B1 and B3 and in the interaction stratifier is `openai/text-embedding-3-small`, also through the gateway.

d: Labels and spend.

Flip labels are single fresh draws under the new version, which matches the estimand: a label answers “would a recompute differ from the cache now?” Labels are therefore noisy themselves. On a three-draw relabeling of the headline pair, that pair's flip rate reads between 6.0% and 7.2% depending on the instrument, and 4.8% of rows change label between two of the labelings. Table S10 reconciles each flip-rate figure quoted for the headline pair and its cached-TRUE stratum with the edit, the cells, and the instrument behind it. The guarantee bounds population rates. In the LLM-as-annotator literature, model labels likewise reach crowd-worker or expert agreement on many tasks, with task-specific noise that has to be validated rather than assumed [16]–[19]. Ledgered model spend across every experiment in this article is \$12.01 for 100,683 billed cells, a lower bound. A further 146,753 cells were computed by probe scripts that call the model directly (the self-flip probes, the majority-vote study, the multi-draw relabeling, the model study, the independence check, and the free-text pilot's draws).

TABLE 3. The strata of the five edit pairs: the cells of each cached value on the $n=2,000$ evaluation vector, and how many of them flip under the edit.

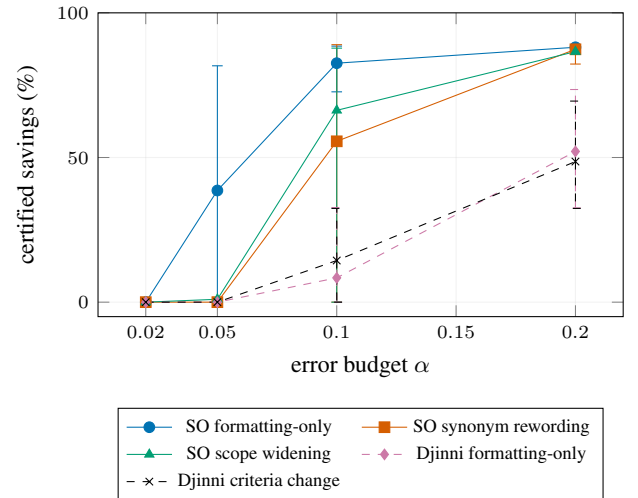
Edit	cached value	cells	flips	flip rate
SO formatting-only	FALSE	1,814	58	3.2%
	TRUE	186	69	37.1%
SO synonym rewording	FALSE	1,825	118	6.5%
	TRUE	175	60	34.3%
SO scope widening	FALSE	1,801	98	5.4%
	TRUE	199	91	45.7%
Djinni formatting-only	Junior	743	54	7.3%
	Mid	559	91	16.3%
	Senior	529	73	13.8%
	Lead/Principal	169	28	16.6%
Djinni criteria change	Junior	738	46	6.2%
	Mid	574	87	15.2%
	Senior	528	89	16.9%
	Lead/Principal	160	27	16.9%

B. MAIN RESULTS

Table 3 gives the strata the certifier decides on, and Table 2 the outcomes. On the Boolean columns the cached-FALSE stratum holds about nine cells in ten and flips at a few percent, while the cached-TRUE stratum flips at a third or more. On the select column the Junior tier flips least, and the other three tiers flip at rates between the two headline budgets. On the stable Boolean column the formatting edit certifies in every replication at $\alpha=0.2$, with 88.1% mean savings, and in nearly every replication at $\alpha=0.1$, with 82.6%. On the main seed its realized presented-cells error is 3.20% at both budgets. At $\alpha=0.05$ it still certifies in 74.4% of replications, but for 38.6% mean savings, because much of the stratum is sampled before the bound clears. The synonym and scope-widening edits certify at $\alpha=0.1$ in most replications and are refused below it, apart from the few scope-widening certificates accounted for below. On the less stable select column both edits certify at $\alpha=0.2$ in every replication, for about half the column, and at $\alpha=0.1$ in most replications, for less than a sixth of it. Of the 20 default-mode configurations, 12 certify in at least one replication. Budgets below a stratum's flip rate are refused, and those refusals are the frontier of Fig. 2. Realized presented-cells error is inside α in every main-seed certification.

a: The α -frontier and adaptive economics.

Savings collapse as α approaches a stratum's flip rate (Fig. 2), as the power analysis predicts; below that rate the certifier refuses. The schedule spends calls only where they can change the outcome: a stratum whose sample rate exceeds the budget is refused at that look, and one near the frontier is sampled further only while its data stay consistent with certification. A single refusal near the frontier is not evidence that the stratum exceeds the budget. The synonym edit at $\alpha=0.1$ is refused on the main seed and certified in 81.4% of the sampling replications, a loss of power that the futility rule accepts in exchange for its early stops.

**FIGURE 2.** The α -savings frontier under the pinned procedure: the mean over 1,000 sampling replications per point, with the 95% percentile interval of Table 2 as the bar, generated from the result files. Savings fall away as α approaches the flip rates of the strata that certify.

b: Noise floors and column stability.

Repeated draws under the unedited prompt give the self-flip rates of the two column types. The Boolean column's floor is 5.0%, measured between pairs of fresh draws over the $n=2,000$ rows. The four-way select's floor is 6.5% on a smaller probe of $n=200$, in which every observed self-flip was between adjacent tiers. At default temperature the Boolean floor reads 9.5% on an $n=200$ two-draw probe whose $T=0$ arm reads 3.0%. The default-temperature floor alone exceeds a 5% budget, which motivates decode pinning. A formatting-only edit flips somewhat more cells than no edit does: on the Boolean column, 7.2% against 5.0% with fresh draws on both sides.

c: Certificates that exceed the budget.

A valid procedure may issue wrong certificates, in at most a δ share of runs. Table 2 contains one configuration where it does: SO scope widening at $\alpha=0.05$, whose certified stratum flips at a rate just above the budget. It certifies in 1.2% of replications. Every one of those certificates is unsafe in the sense of Theorem 1, and that share sits inside the per-stratum $\delta_j = \delta/K = 0.05$ of that column. This is an exact test spending part of its δ_j on a stratum just past the budget, the behavior the calibration study measures at scale (Section V-F). The reuse-set rate, which the default mode does not certify, exceeds α on some default-mode certificates in 3 configurations (the exceed. column of Table 2).

d: The strict mode.

The strict mode guarantees the reused cells alone, at a threshold that shrinks as sampling deepens. That stronger guarantee is nearly free where a stratum's flip rate is well below the budget and expensive near it. On the headline edit it costs nothing at $\alpha=0.2$ and two points of mean savings at $\alpha=0.1$

(80.6% against 82.6%). At $\alpha=0.05$ it certifies in 4.5% of replications, where the default mode certifies in 74.4%. On the select column it cuts mean savings by more than a third at $\alpha=0.2$ and certifies in at most 2.7% of replications at $\alpha=0.1$. Appendix G tabulates both modes at the three budgets where either certifies.

e: A graded relation (retrospective).

After the exact-match results were in, we re-scored the stored labels of the seniority column's formatting edit under a tolerance: a cell flips only when the fresh tier is more than one step from the cached one. The pinned procedure then certifies it in every replication at $\alpha=0.05$, a budget that exact match refuses, for 66.3% mean savings. This is a weaker guarantee, not a tighter one: it says nothing about moves of one tier, and it would be the wrong relation for the column's criteria edit, whose purpose is to move some candidates up one tier (Appendix I).

C. BASELINES

Four comparisons isolate what sIVM adds. Table 4 lines the policies up, and Appendix H gives the detail of each.

(B1) *Verified-cache similarity has no cross-version signal.* The per-row similarity between a row's two instantiated prompts, the quantity a verified semantic cache thresholds, predicts cell flips at chance. The area under the receiver-operating-characteristic curve (AUC) is 0.454 on the formatting edit and 0.499 on the widening edit. Adding "a version key" to such a cache therefore cannot produce a sound cross-version policy. Our interaction score carries signal when the edit is semantic (AUC 0.669 on widening) and none when flips are noise-driven (0.406 on formatting). That is why the cached *value* carries the stratification in the formatting case.

(B2) *Aggregate-only certification*, the pinned procedure without strata, is valid for its aggregate estimand and silently corrupts subgroups. At $\alpha=0.2$ on the formatting edit it certifies 97.8% of the cells at 6.39% aggregate error, while the cached-TRUE cells it reuses carry 37.2% error. Per-stratum bounds are what make the guarantee mean something for every user-visible slice of the table.

(B2') *A SUPG-style competitor, matched.* The closest existing method is proxy-certified selection [11]: score rows with a cheap proxy, spend an oracle budget labeling a sample, and threshold the proxy so that the selected set's error clears the target. We transported it to maintenance with the cached value itself as its proxy, the reduction Section III describes. We matched it to sIVM on the oracle calls, the labels, the error target, the bound, and the total failure probability δ . Matched this way it wins on volume and loses on where its errors land. At $\alpha=0.2$ it reuses nearly every cell on both edits and satisfies the aggregate bound it was given. The extra volume is the volatile stratum: 183 cached-TRUE cells on the formatting edit, from a stratum whose floor is 23.0%. sIVM reuses none of them. An aggregate target is satisfied by any mixture that averages out, so it buys volume with the correctness of the least stable subgroup. The result does not

depend on the choice of proxy, nor on the competitor's looser per-test level (Appendix H).

(B3) *Guarantee-free embedding transfer* (GPTCache-style [20], $\tau=0.95$), the deployed status quo, reuses every cell on the formatting edit and 84.1% of them on the widening edit, absorbing error at about the population flip rate. Its threshold controls volume, never error. A sweep of τ takes the reused share from nearly everything to nothing within a window about 0.03 wide, while the error among reused cells stays at the population flip rate (Appendix H). The trivial policies, always recompute and always reuse, bracket the design space in Table 4. sIVM is the only row whose reused cells carry a per-stratum bound.

D. OTHER MODELS

Every measurement above uses one model. We therefore ran the lab column's two pairs on 6 further low-cost models from four other providers (Table 5), and the remaining three pairs on 5 of them (Table S11). The runs use the same seeded rows as the primary evaluation vector, the same prompts, $T=0$, and one fresh draw per cell. Each model's cache is its own draw under the old version, and its floor is measured against a second draw of that version. A flip is therefore the model disagreeing with itself across the edit. The two tables report the replication analysis of Table 2, repeated on each model's stored draws.

a: What transfers.

Every model has a measurable self-flip floor on the lab Boolean column, from 0.4% to 34.8% (Table 5). On the production column one model, Gemini 2.5 Flash-Lite, shows no self-flip in its rows (Table S11). Certified reuse is not a property of the primary model. On each of the 6 further models at least one edit certifies in at least 99.8% of replications at the loose budget. Unsafe certificates stay rare. Across the 54 entries of the two tables, a certificate for a null is issued in at most 1.5% of an entry's replications, inside the per-stratum level of every column. A realized presented-cells error above the budget occurs in at most 0.7%.

b: What does not transfer.

Which edits are benign does not transfer. The formatting-only edit is the benign one on the primary model. It certifies in every replication on Gemini 3 Flash and Gemini 3.8 Flash, and in almost none on Gemini 2.5 Flash-Lite and GPT-5 nano. On GLM-4.7-Flash it certifies only a small stratum (3.3% mean savings). On Gemini 2.5 Flash-Lite it flips 25.45% of cells against a floor of 4.85%. There and on GLM-4.7-Flash it moves the share of TRUE answers, in opposite directions, which self-disagreement alone would not do, so on those two models the edit shifts the model's decision (Appendix J). On the three models where the formatting edit fails, the synonym rewording certifies a stratum in most replications at the loose budget instead, because its flips concentrate in one cached value and leave the other reusable. The ordering that the edits' labels suggest (formatting benign, rewording not) inverts on

TABLE 4. Reuse policies on the SO formatting and scope-widening edits at $\alpha=0.2$ ($n=2,000$, main seed). Realized error is the flip rate among reused cells; the trivial policies' errors follow by construction (always-recompute has none; always-reuse carries the population flip rate). B2 additionally reports the error of its reused cached-TRUE subgroup in parentheses. B2' is matched to the 90 labels sIVM spent; B2 runs the pinned schedule without strata and stopped after 45 labels on the formatting edit (360 at $\alpha=0.1$) and 90 on the widening edit.

Policy	Guarantee	Formatting		Widening	
		reused	error	reused	error
Always recompute	none needed (zero error)	0.0%	0.0%	0.0%	0.0%
Always reuse (status quo)	none	100%	6.35%	100%	9.45%
B3 GPTCache, $\tau=0.95$	none	100%	6.35%	84.1%	9.8%
B2 aggregate-only	aggregate	97.8%	6.39% (37.2%)	95.5%	9.37% (44.6%)
B2' SUPG-style, matched	aggregate	95.5%	6.54%	95.5%	9.58%
sIVM (this work)	per stratum	88.5%	3.28%	87.8%	5.47%

some models and holds on others. Benignness is a property of the (edit, model) pair, and the certifier measures it per stratum without consulting the label.

c: Floors are per stratum.

The floor binds for the identity edit only (Section II), and some strata of this study flip less under an edit than under none. The two clearest are on the lab column. Under the formatting edit GLM-4.7-Flash's cached-FALSE stratum flips 17.0% against an identity floor of 24.4%, and GPT-5 nano's cached-TRUE stratum flips 24.3% against 44.9%. GPT-5 nano's pooled floor, 34.8%, sits above every budget in the grid. Its one certifying stratum on the lab column sits below that pooled figure for the same reason: a floor is a per-stratum quantity, measured under the first version's prompt.

E. DEPLOYMENT SCALE

To run the certifier at the scale of a full corpus, we materialized the Boolean lab column across all 89,184 rows. Here "deployment scale" means a script running the certifier the table uses over the full corpus, with no outside users involved. The materialization came to 78,956 billed cells in a single ledgered run costing \$9.31, over 7.2 hours of wall clock at 32-way concurrency, and the remaining 10,228 rows cost nothing. We then certified the formatting edit against that column, with the real model as oracle, at three budgets ($\alpha=0.2$, 0.1, and 0.05) on two model snapshots over one seeded sample path (Table 6). The first certification uses the oracle draws of the live run of August 2026. The second was drawn live on 29 September 2026, 55 days later, against the same cache. The budgets share the path at no cost in δ , because the per-look bounds do not depend on α . The August run itself certified under the Maurer–Pontil bound, which Theorem 1 does not cover, and the system applied that certificate. Table 6 therefore replays the August draws under the exact bound. Those rows are retrospective: the bound was fixed after the draws had been seen. The September rows were drawn live under the exact bound, fixed beforehand (Appendix C gives the certificate applied in August).

a: The August draws, replayed.

At the loose budget the exact bound certifies the cached-FALSE stratum at look 45. The whole certification costs **90 oracle calls** and reuses **81,424 cells**, 91.3% of the column. The ceiling is 91.3%, set by the cached-TRUE stratum the certifier never certifies. The procedure thus captures nearly all the reuse this cache offers (Section II). The tight budget, $\alpha=0.1$, costs 225 calls. At $\alpha=0.05$ the stored draws run out before the schedule's deepest look. Drawing the missing cells later would mix two snapshots, so the August evidence is silent there. Audited against the oracle-computed cells of that snapshot, the reused rows flip at 3.15% (89 of 2,821 audited rows). A separate verifier script recomputes every audit of Table 6 from the row identifiers each run persisted and the stored cells.

b: The September run: drift, measured.

Assumption 1 scopes a certificate to the model snapshot its oracle draws came from, so we reran the certification live on 29 September 2026 with the same model identifier, gateway, prompts, cache, and seeded path. The snapshot had moved. On the 720 cached-FALSE rows the August run had sampled, the edit's flip rate against the cache rose from 3.5% to 6.9% (McNemar $p < 0.001$). We cannot tell a change of the provider's model from a change of its default settings or of the gateway's routing; to the user they are one event, and the certifier measures their sum against the cache it is asked to reuse. The loose budget certified again at look 45, for 90 calls, in 101 seconds of wall clock including the oracle calls. The tight budget needed 405 calls, one look deeper than on the August draws. The futility rule refused $\alpha=0.05$ after 135 calls, correctly: the stratum's flip rate against the aged cache exceeds that budget. Audited against every September oracle cell, the reused rows flip at 7.46% under the loose certificate and 7.96% under the tight one, inside both budgets and well above the August figure. Appendix C gives the like-for-like drift comparison, the per-look evidence, and the run's accounting.

TABLE 5. Further models on the lab column's two edit pairs under the pinned procedure (presented-cells estimand, $\delta=0.1$; the same seeded rows as the primary evaluation vector, one fresh draw per cell, $T=0$). Each entry gives mean savings with the certification rate in parentheses, both in percent over 1,000 sampling replications of that model's stored draws. The primary row repeats Table 2, with the column-level floor of the noise-floor paragraph. Here n is the rows requested; a few requests per model failed, leaving at least 1,966 usable rows per measurement. A rate that falls half way between two one-decimal values is printed to two decimals. Gemini 3 Flash ran these two pairs only: in its reasoning regime the remaining pairs would have been the costliest runs of the study, and they were left out.

Model	n	floor	formatting-only (v1→v2)			synonym rewording (v2→v3)		
			flip	$\alpha=0.1$	$\alpha=0.2$	flip	$\alpha=0.1$	$\alpha=0.2$
DeepSeek V4 Flash (primary)	2,000	5.0%	6.35%	82.6 (98.9)	88.1 (100.0)	8.90%	55.6 (81.4)	87.4 (99.9)
Gemini 2.5 Flash-Lite	2,000	4.85%	25.45%	0.0 (0.0)	0.1 (0.1)	31.5%	62.1 (100.0)	63.0 (100.0)
Gemini 3 Flash	2,000	0.4%	4.5%	82.1 (100.0)	83.2 (100.0)	6.2%	80.7 (99.2)	85.3 (100.0)
GLM-4.7-Flash	2,000	10.35%	25.0%	0.0 (0.0)	3.3 (70.7)	31.3%	0.7 (1.3)	54.5 (98.7)
Qwen3.7-Flash	2,000	5.45%	14.35%	0.5 (1.4)	51.5 (99.3)	27.2%	40.0 (94.5)	48.8 (100.0)
GPT-5 nano	2,000	34.8%	39.6%	0.0 (0.0)	0.0 (0.1)	39.7%	0.0 (0.0)	21.9 (65.4)
Gemini 3.8 Flash	2,000	0.95%	0.85%	92.6 (100.0)	95.5 (100.0)	4.9%	90.3 (100.0)	90.5 (100.0)

TABLE 6. Deployment-scale certification of the formatting edit on the 89,184-row column under the exact bound: one seeded sample path, two model snapshots. "FALSE look / flips" is the look at which the cached-FALSE stratum was decided and the flips observed there. The cached-TRUE stratum is refused at its first look throughout. Audited error is the flip rate among reused rows that hold an oracle cell of that snapshot.

Budget	oracle calls	FALSE look / flips	reused	savings	audited error (flips/rows)
<i>August 2026 snapshot: the stored oracle draws of the live run, replayed</i>					
$\alpha=0.2$	90	45 / 1	81,424	91.3%	3.15% (89/2,821)
$\alpha=0.1$	225	180 / 5	81,289	91.1%	3.16% (85/2,686)
$\alpha=0.05$	not reachable from the stored draws (699 further draws needed)				
<i>September 2026 snapshot: oracle drawn live, same cache</i>					
$\alpha=0.2$	90	45 / 2	81,424	91.3%	7.46% (104/1,395)
$\alpha=0.1$	405	360 / 20	81,109	90.9%	7.96% (86/1,080)
$\alpha=0.05$	135	90 / 6 (refused)	0	0.0%	—

c: Sample cost against table size.

The schedule stops on evidence about a stratum's rate, so the sample is bounded whatever the table's size, while the population a certificate covers grows with the table. At $\alpha=0.2$ the certification spent 90 calls on 89,184 rows, where the benchmark's replications spend 90–180 calls (mean 97) on 2,000 rows. Savings therefore rose from 88.5% on the benchmark's main seed to 91.3% at the scale of the corpus. Savings here are shares of rows; rows with identical bound content share one stored cell (Appendix C).

F. VALIDITY CHECKS

a: Calibration.

The calibration study plants strata at known flip rates and runs the certifier on them (Fig. S1): 198 configurations over planted rates $p \in [0, 0.25]$, budgets $\alpha \in \{0.05, 0.1, 0.2\}$, three stratum sizes, and both modes, 396,000 runs in all. The worst rate of unsafe certificates, the quantity Theorem 1 bounds, is 1.00%, against the study's per-stratum level $\delta_j=5\%$. Planting by rate is not the least favorable case, because a planted stratum's realized count often falls on the safe side of the budget. The least favorable case is a fixed population one flip past the budget, where the probability of an unsafe certificate can be computed exactly. Over the 9 (budget, size) combinations of the grid it is at most 2.7% in the default mode and 1.1% in the strict mode, inside δ_j in each. These figures assume draws that are independent across rows;

the next paragraph prices dependence. The gap to δ_j comes from the Bonferroni split across looks, the discreteness of the test, and the futility rule. Appendix F gives the computation, the argument that this population is the least favorable one, and the value for each combination.

b: Independence across rows.

Assumption 1 asks that oracle draws not interfere across rows, and provider-side batching could in principle correlate concurrent requests [8], [9]. Against the same cache, the formatting edit's new version was drawn for each of 300 rows with one request in flight and again at a concurrency of 32. A further concurrent arm of 1,000 rows was drawn a day later. Appendix D gives every test; three findings matter here. First, among completed requests the flip rates agree, 8.9% sequential against 8.8% concurrent, a comparison that could detect a difference of 7.0 percentage points, not a smaller one. No lag-one test of serial dependence rejects. Second, failed requests tracked the serving conditions more than the concurrency: the first concurrent arm had 28 against the sequential arm's 9 of 300, and the larger arm, under faster serving, had none. Counted as flips, as the certifier counts them (Section IV), the arms read 11.7%, 17.3%, and 7.1%, with an exact McNemar $p=0.036$ for the paired two. The rule is conservative, so failures can cost certificates but cannot create them.

Third, dependence among draws issued together is not

excluded, and the planned test for it is confounded. Flip counts over blocks of completions are overdispersed in the first concurrent arm (2.08 times the binomial variance) but also in the sequential arm on the same rows (1.39), so the excess follows the shared rows at least as much as the concurrency. The larger arm, on other rows, reads 1.26. Pooled over both concurrent arms the ratio is 1.42 (95% interval [0.94, 2.37]), and the one-sided test rejects narrowly. Priced as if all of it were dependence, on a stratum one flip past the budget, the certifier errs with probability at most 2.3% under independence, 5.4% at the pooled estimate, and 13.1% at the interval's upper end, against a nominal δ_j of 0.05. The first concurrent arm alone, eight blocks under slow serving, is compatible with a much larger figure. The check therefore supports less than independence: it does not show that concurrent draws interfere, and it cannot rule out an error probability two to three times the nominal one on a stratum just past its budget. That exposure is confined to such strata, and at $\alpha=0.2$ the deployment stratum's audited error is far below its budget. The replication results resample frozen labels, a fixed population for which the hypergeometric law holds however the labels were drawn, so only a live certification relies on the assumption. Its few calls can be issued one at a time, at a cost in wall clock and none in calls, although the sequential arm is too short to show that no dependence remains then. The check covers one model through one gateway at one concurrency, in the September snapshot, whose certificates were themselves drawn concurrently.

G. ABLATIONS

a: The bound ablation.

The bound, more than the stratification, sets the frontier. We swapped it inside the same pinned procedure over the same stored labels (Table S1; Appendix B gives every configuration). The five bounds are the exact bound of Eq. (1), its binomial limit (the Clopper–Pearson bound [12]), the Maurer–Pontil empirical-Bernstein bound [13], the Bardenet–Maillard without-replacement bound [14], [21], and a Waudby-Smith–Ramdas betting confidence sequence [22], which needs no split across looks. Across the 20 configurations of Table 2 (main seed), the exact bound certifies 10 of them, against 7 under Maurer–Pontil and 7 under Bardenet–Maillard, at higher mean savings and on fewer oracle calls. The empirical-Bernstein bounds pay a range term that a binary loss does not need. The betting sequence beats the exact bound on 1 configurations, where the split across looks bites hardest, and loses to it on 2; its validity for draws without replacement rests on a caveat that Appendix B states. Replayed through the calibration protocol, the exact bound's rate of unsafe certificates at planted nulls is at most 1.00% against a per-stratum δ_j of 5%. The exact bound is the pinned one because the sampling model itself implies it: it uses the hypergeometric law with no relaxation.

b: The stratification hierarchy.

One stratum is too few: aggregate-only certification conceals the cached-TRUE subgroup's error (baseline B2). More strata than the cached values are usually too many: refining each value stratum by embedding-interaction tertiles keeps validity (Assumption 2) but costs power through the δ/K split unless within-value risk is heterogeneous. On the headline configuration the refinement certifies less than value strata alone (82.7% against 88.1% mean savings on SO formatting at $\alpha=0.2$). Across the 40 shared configurations of the released ablation it is better in 6 and worse in 22, and every win sits at a budget where value strata alone certify in at most 2.7% of replications. We report this as a negative result.

c: Minority-stratum anatomy.

The cached-TRUE stratum of the Boolean column flips 37.1% under a formatting edit. One candidate cause is cache noise, which a majority vote of three draws at write time would remove [23]. The stored draws do not support it. Voting on both sides (vote-of-3) brought the stratum's flip rate to 24.7% in a dedicated probe and to 34.4% on the three-draw labels, two separate runs over the same 186 rows. The vote overturned 20.4% and 16.7% of the cached TRUE values, respectively. A cache built by the vote fares no better. Its TRUE stratum, the 188 rows whose three-draw majority is TRUE, has a lower floor (17.6% against 23.0%) but flips at 41.0% against fresh draws of the edited prompt (Table S10). Voting at write time cleans the cache without making the minority stratum certifiable under this edit. Refusing and recomputing that stratum is the response the data support.

VI. THE PROMPT-EDIT BENCHMARK

Semantic benchmarks exist, but they benchmark *engines* on a fixed task [24]; none exercises maintenance across prompt versions, which needs a different artifact: version pairs plus per-cell flip labels against the new version. Ours comprises five components. The first is two redistributable corpora: the Stack Overflow Developer Survey (89,184 rows, ODbL 1.0 with the Database Contents License, DbCL 1.0) and Djinni English candidate profiles (210,250 rows, MIT). The second is three materialized columns of two types: two Boolean columns over structured facets, one of them the production column and one a lab replica seeded from its second version; and one four-way ordinal select over long free text. The third is five realistic edit pairs spanning formatting-only edits, synonym rewording, scope widening, and a criteria change. The fourth is per-cell flip labels for $n=2,000$ rows per version pair at pinned decode, the three-draw relabeling of the headline pair (which also supplies its self-flip floor), and the select column's self-flip probe. The labels and the relabeling draws are 28,000 labeled cells in all: 16,000 distinct version cells and 12,000 relabeling draws. The fifth is the deterministic harness (seeded sampling, adaptive certifier, baseline policies) that reproduces every number in this article. Table S12 lists the five pairs with their exact deltas. Ledgered model spend across the entire evaluation is \$12.01 (plus 146,753

cells from direct-call probes the ledger does not see), so the benchmark is cheap to extend to other low-cost models.

VII. RELATED WORK

Table S13 positions sIVM against the lines of work it borrows from, is compared with, or could be mistaken for, by the quantity each certifies and the axis of change it holds fixed.

a: *Guaranteed semantic query processing.*

Semantic-operator systems bound the error of *one-shot* execution against a reference model. They include LOTUS's optimized operators with accuracy targets [25], SUPG's proxy-based selection with precision/recall guarantees [11], and its aggregation counterpart ABae, which stratifies by proxy before spending an expensive predicate [26]. They also include BARGAIN's adaptive cascade thresholds [27], ThalamusDB's approximate natural-language predicates [28], and pipeline-level accuracy budgeting in Task Cascades [29]. All certify a *fixed* definition per query; none transfers evidence across a definition change. We reuse their statistical framing (a labeled sample, a bound, a threshold) in a maintenance loop those systems do not model. Because the loss is binary and the sample is drawn without replacement from a frozen stratum, the hypergeometric form of Clopper and Pearson's limits [12] is available, and it dominates the empirical-Bernstein bounds a general bounded loss calls for (Section V-G). Time-uniform confidence sequences [22], [30] would remove the split across looks. The certificate itself is distribution-free risk control in the sense of risk-controlling prediction sets [31] and Learn then Test [15], evaluated per stratum and per look with the multiple-testing accounting those frameworks make explicit. Declarative LLM data systems that optimize cost and quality without a statistical target (Palimpzest [32], DocETL [33], Evaporate [34], and UQE [35]) compute the same kind of prompt-defined per-record outputs, and none certifies which of them survive a redefinition.

b: *Sampling inspection and sequential tests.*

Statistically, a certification is a lot-acceptance decision by attributes [36]. A stratum is a finite lot and a flip a defect, so the exact bound is the hypergeometric calculation of acceptance sampling. The look schedule is a group-sequential design that splits the failure probability evenly across the looks, where the spending functions of that literature [37], [38] would allocate it unevenly. Risk-limiting election audits [39] share the shape of the whole procedure: sample until the evidence bounds the risk of a wrong outcome, and otherwise fall back to the full count, as a refusal falls back to recomputation.

c: *Verified caching.*

vCache certifies that a *new query* may reuse a cached answer under one fixed definition, with learned per-entry thresholds and user error rates [40]. Evergreen spends confidence sequences and early stopping to cut the cost of *verifying* semantic aggregates, not to decide reuse [41]. FreshCache, the closest in shape, approves a cache hit only when a learned

staleness model puts the probability that the entry is stale below a per-tier error budget [42]. It differs on the two axes that decide the guarantee. Its risk is *change-of-world* (open-web evidence aging under fixed query semantics) where ours is *change-of-definition*, and its per-tier quantity is a fitted model's point estimate where ours is a finite-sample upper confidence bound. Across a definition change the query-to-entry similarity that such caches threshold carries no per-cell flip signal (baseline B1). The two axes compose: a certificate is scoped to a snapshot, and change-of-world machinery decides when the snapshot itself expires.

d: *Incremental and adaptive view maintenance.*

Sampling a stale materialized view to bound the error of answers served from it [43] is the closest classical relative, and the difference is what goes stale: there the data drifts under a fixed view definition, here the definition changes over fixed data. The look schedule is online aggregation's template [44], a running estimate whose interval tightens as sampling proceeds, repurposed to certify a stratum. Classical incremental view maintenance (IVM) propagates base-data changes through a fixed view definition [45], and view adaptation after redefinition [46] rewrites the materialization when the definition changes. Both assume deterministic operators, where a rewrite either preserves a subexpression or does not. Stochastic LLM cells break that premise. Even a semantically null rewrite flips cells at about the column's noise floor (6.35% against 4.5%, both measured against the stored cache), so "preserved" is not a property a rewrite can have here, only a rate one can bound. Streaming semantic systems recompile operators mid-stream but recompute outputs instead of certifying reuse of old ones [47], [48], and streaming cascades certify on the *data-arrival* axis and leave the definition axis open [49]. sIVM is, to our knowledge, the first maintenance procedure whose unit of transfer is a *statistically certified set of stochastic cells*.

The nearest counterexample in shape is step-level reuse with lightweight verification and selective patching across perturbed requests [50], which reuses computation when a request changes. Its verification is a deterministic per-step check, so it carries no error budget and makes no population statement about the set it reuses. In-database prompt management [51] and versioned, provenance-tracked prompt views [52] supply the substrate this work assumes, not a competing mechanism. We take the framing of prompts as versioned views from that line and claim only the certification machinery built on it, whose reuse is of *outputs*, not of a model's internal state.

e: *Prompt sensitivity and regression.*

Prompt-delta analysis for assertion synthesis [53] anticipates our edit taxonomy but targets pipeline quality checks, not reuse certification, and interactive LLM regression tools frame migration pain without guarantees [54]. Our finding that surface edit class can fail to predict flip rate (not how often it does) shows why measurement, not taxonomy, must

carry the decision. Formatting changes that preserve meaning are known to move task accuracy widely, and differently across models [55]; our per-cell flip rates, and their variation across models, are that sensitivity seen at the level of individual outputs. A served model can also change under a fixed name [56], which is the drift our September re-certification measured against a cache.

VIII. DISCUSSION AND LIMITATIONS

a: Which budgets are usable, and for what.

Most of the savings sit at $\alpha=0.2$ and $\alpha=0.1$, and neither is a small tolerance (Table S7 sets out what each budget tolerates and buys). A budget is a share of a certified stratum. On the running example the certified stratum is the cached-FALSE one, so the cells at risk are hidden positives: rows shown as FALSE that a recompute would show as TRUE. At the scale of the corpus the loose budget tolerates 16,293 of them, about twice the 7,715 rows the column marks TRUE at all, and the tight budget still tolerates more than that number. The audits found fewer, but not few. Carried to all the reused cells, the error rates audited under the loose certificates come to about 2,600 hidden positives on the August draws and about 6,100 in September. The audits are samples, and their 95% confidence intervals carry to 2,100–3,100 and 5,000–7,300. Each is a row on which a recompute would differ, which need not mean the cached value is wrong, and a certificate promises only its budget.

A budget that loose is defensible where a missed positive is cheap and the column is advisory: a lead list or an enrichment sheet that a person keeps working through, or an exploratory filter recomputed before anything is decided on it. The alternative there is today's status quo, which reuses everything at the population flip rate with no bound at all. It is not defensible where a FALSE closes a case (a screen that rejects on it), or where the column feeds an aggregate or a downstream filter. There a tighter budget and the strict mode are the right setting, and Table S7 shows what they buy. At $\alpha=0.05$ the stable Boolean column still certifies in most replications, but for less than half the savings, and the strict mode almost never. The select column certifies nothing below $\alpha=0.1$, and at the scale of the corpus nothing below $\alpha=0.1$ certified. Tight budgets are affordable in calls, because the sample is bounded (405 calls at $\alpha=0.1$ in September, against 89,184 cells). What they cannot buy is certification of a stratum whose flip rate is near or above them. A practitioner should therefore read the frontier of Fig. 2 against the column's floor, and choose the budget the downstream use tolerates, not the one that maximizes savings.

b: Statistical scope.

Bonferroni across strata and looks is sound but conservative (Section V-G prices it), and the exact bound leaves a gap between its measured unsafe-certificate rate and the nominal δ . The without-replacement betting sequence of [22] or a time-uniform boundary [30] would keep the exact bound's validity while removing the split across looks. A group-sequential

calibration of the six looks [37], [38] is a third route. The finite sum behind the calibration study's worst-case figures gives the exact error probability of any per-look levels, so the levels could spend nearly all of δ_j , at the price of reading only one budget from a sample path. Selection by e-values (the e-value Benjamini–Hochberg procedure [57]) composes with any of these. None of them is needed for the guarantees stated here. The default mode is also conservative for its own estimand, since certifying when $u_{j,t} n_j - k_j \leq \alpha n_j$ would bound it as well; we keep the simpler rule, which bounds the whole-stratum count too. The independence of oracle draws across rows is an assumption that our check probes but does not establish (Section V-F), so a live certification under concurrency inherits whatever dependence the serving path introduces. Our stratifiers are first instances: the lexical variant is weak on broad edits, and the embedding variant carries signal only on semantic ones. Learned interaction models, or a cheaper model's answer used as a proxy, are an open lever with guaranteed validity (Assumption 2).

c: Sequences of edits.

Reuse is one edit deep in the released system: a cell reused under a certificate is recomputed at the next edit (Section IV), so savings do not accumulate over a series of edits. Theorem 1 does not force that choice, because the guarantee concerns the stored values whatever version computed them, so a later certification could test carried-forward values against fresh draws of the newest version. We have not evaluated sequences of edits, and each certification in a sequence spends its own δ .

d: Equivalence and columns.

In the main evaluation equivalence is exact match, and the columns are two Boolean columns and one four-way ordinal select; numeric and free-text columns, which the introduction's product examples also serve, are not evaluated there. The five author-written edit pairs, one per class per column, show that surface class can fail to predict flip rate, not how often it does. Because both Boolean columns ask one question over one corpus, the observation that a column's own stability dominates the edit rests on two columns. The equivalence relation is the user's to state. Theorem 1 needs only that it yield a binary flip indicator, so a graded column can take a tolerance in place of exact match (Section V-B measures one on the seniority column), and a numeric column would take one the same way. A free-text column needs a judge, and the judge's own error enters the budget. Let $\hat{m}$ be an upper confidence bound at level δ_{cal} on the judge's miss rate among the pairs it calls equivalent, calibrated on human-labeled pairs of the same column. Certifying at $\alpha - \hat{m}$ bounds a certified stratum's true rate by α with probability at least $1 - \delta - \delta_{\text{cal}}$, provided the judge misses no more often in the certified stratum than in the calibration pool. Appendix E gives the argument.

e: A free-text pilot.

We ran the recipe once, on a free-text column over 2,000 SO profiles that names each respondent's primary technical specialty, with a formatting-only edit and a synonym rewording. The judge is the primary model, and its miss rate was calibrated on 201 pairs labeled by the author. The result is negative on power: under the judge the column's own floor, two draws of the same prompt, is 40.1%, above every budget, and the certifier refused every stratum of both edits. The calibration is clean on the author's labels: the judge missed 0 of the 150 pairs it called equivalent, which takes 2.0 percentage points off each budget. It is not clean on a second model's labels of the same pairs, and one human labeler cannot settle which reading is right. The machinery carries over to a judge unchanged, and a judge strict enough to calibrate cleanly can cost all the power (Appendix E).

f: Models and snapshots.

One low-cost reasoning model carries the full experimental program. On 6 of the 6 further models at least one edit certified in nearly every replication at the loose budget, while the same edit's flip rate ranged widely (Section V-D). The quantitative frontier (where savings sit for a given α , and which edits are benign at all) is therefore specific to the (edit, model) pair. The qualitative structure held on every model: a floor that is measurable on the lab column, which limits what certifies when an edit leaves the output distribution unchanged, and a certifier whose refusals track the measured flip rate. Certificates are scoped to the model snapshot their oracle draws came from, and Section V-E shows what that scope is worth: the same identifier had drifted 55 days after the deployment run, and the certifier measured the drift and moved its certificates with it. Detecting when a snapshot has moved, without a full re-certification, i.e., composing sIVM with change-of-world invalidation [41], [42], is the natural next step.

g: Economics.

What matters economically in production is how long a column takes to become consistent again under provider rate limits. Certified maintenance samples 5%–74% of cells across the configurations of Table 2 that certify in most replications (5% on the headline edit at $\alpha=0.2$) and recomputes only the strata it refuses. That is a wall-clock and rate-limit gain that grows with table size, because sample sizes do not; the live re-certification of Section V-E took 101 seconds for the loose budget. This article does not measure two things. The certificate surface in the UI was not studied with users, so whether a person acts differently on a column that shows one is open. No claim here depends on the answer: the guarantee is a statement about the cells, whatever the interface shows, and a user study is human-subjects work of its own. The cost figures come from one ledgered materialization and one priced maintenance replay at the gateway's list rates, so they show only the scale of the saving.

IX. CONCLUSION

Prompt-defined columns are materialized views whose *definition* changes far more often than their data, and the systems serving them today choose between recomputing everything and reusing everything. sIVM shows the middle is reachable and cheap: freeze a stratification, sample on a doubling schedule, bound each stratum's flip rate with an exact finite-population bound, reuse only what clears the budget. At the scale of a full corpus and a budget of $\alpha=0.2$, that cost 90 oracle calls to certify reuse of 81,424 of 89,184 cells. The sample did not have to grow with the table, because the schedule stops on evidence about a *rate*, at a cost bounded independently of the size. These results come from one primary model on Boolean and select columns. On 6 of 6 further models, from four providers, at least one edit certified, and which edits did so varied by model.

Beyond reuse, the certifier diagnoses the column it maintains. A stratum refused at every budget is a slice of the table on which the model does not reliably reproduce its own answers, identified by a futility stop at the cost of one look. A flip rate that has risen on the same rows against the same cache 55 days later is a model that has moved under it, and the certifier reports that movement in the currency the user pays, oracle calls per certified budget.

AVAILABILITY AND REPRODUCIBILITY*a: What is released.*

The system, the certifier, every experiment script, and every result artifact are public in a single repository, <https://github.com/arjunlohan/project-lore>, under the Poly-Form Noncommercial License 1.0.0, a source-available license that permits research and other noncommercial use. The release contains ingestion scripts for the two corpora, the five versioned edit pairs, the per-cell flip labels, the certifier, the adapter conformance suite, and every script and result artifact that produces a number in this article. It also holds the cached and oracle values of the 3,520 rows the deployment audits read. The corpora are the 2023 Stack Overflow Developer Survey [58] and the English candidate profiles of the Djinni Recruitment Dataset [59], both public and redistributable (under ODbL 1.0/DbCL 1.0 and MIT) and used as distributed by their maintainers. The Djinni corpus is anonymized at source; this work collects no new human-subject data, attempts no re-identification, and releases only per-cell labels and aggregate statistics. Rerunning the evaluation needs a model endpoint of one's own; the released labels reproduce the statistical results without one.

b: Generated numbers.

Every quantitative claim in this article's prose, tables, and figures is emitted by a single generator (`scripts/experiments/gen-paper-assets.ts`) from the result artifacts and the system's cost ledger, and cited through macros, never typed inline. The numbers computed from the running system's database (the per-stratum floors, the ledger economics, the materialization wall clock, the

certificate's stamp count) are dumped to an artifact, so each can be checked against the release without our instance; regenerating them requires the populated database. A checker (`scripts/check-paper-numbers.ts`) enforces the macro discipline and fails on ten classes of defect, listed in Appendix A.

c: Hazards and the preprint.

Three reproducibility hazards we hit are documented in the harness and in Appendix C. An earlier version of this article is public as a preprint (Zenodo, doi:10.5281/zenodo.21833641); it is "the preprint" wherever this article says so. Relative to it, the certifier's default bound changed from Maurer–Pontil to the exact finite-population bound and every number was regenerated. The preprint's certification results are reproducible from the same release with the Maurer–Pontil arm selected.

ACKNOWLEDGMENT

The author used Claude (Anthropic) as an editing assistant for the manuscript text and as a coding assistant for the system and experiment harness. All technical content, experimental designs, measurements, and conclusions were specified, reviewed, and verified by the author.

REFERENCES

- [1] Google LLC, "Use the AI function in Google Sheets," Google Docs Editors Help, 2026, accessed 29 September 2026. [Online]. Available: <https://support.google.com/docs/answer/15877199>
- [2] Airtable, Inc., "Using Airtable AI in fields," Airtable Support, 2026, accessed 29 September 2026. [Online]. Available: <https://support.airtable.com/articles/8052242094-using-airtable-ai-in-fields>
- [3] Notion Labs, Inc., "Notion AI autofill for databases," Notion Help Center, 2026, accessed 29 September 2026. [Online]. Available: <https://www.notion.com/help/autofill>
- [4] Snowflake Inc., "Snowflake Cortex AI functions: AI_COMPLETE and AI_CLASSIFY," Snowflake Documentation, 2026, accessed 29 September 2026. [Online]. Available: https://docs.snowflake.com/en/sql-reference/functions/ai_complete
- [5] Microsoft Corporation, "COPILLOT function," Microsoft Support, 2026, preview feature, withdrawn 14 September 2026; accessed 29 September 2026. [Online]. Available: <https://support.microsoft.com/en-us/excel/functions/copilot-function>
- [6] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An empirical study of the non-determinism of ChatGPT in code generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 42:1–42:28, 2025, arXiv:2308.02828, doi:10.1145/3697010.
- [7] B. Atıl, S. Aykent, A. Chittams, L. Fu, R. J. Passonneau, E. Radcliffe, G. R. Rajagopal, A. Sloan, T. Tudrej, F. Ture, Z. Wu, L. Xu, and B. Baldwin, "Non-determinism of 'deterministic' LLM system settings in hosted environments," in *Proc. 5th Workshop on Evaluation and Comparison of NLP Systems*, 2025, pp. 135–148, doi:10.18653/v1/2025.eval4nlp-1.12.
- [8] J. Yuan, H. Li, X. Ding, W. Xie, Y.-J. Li, W. Zhao, K. Wan, J. Shi, X. Hu, and Z. Liu, "Understanding and mitigating numerical sources of nondeterminism in LLM inference," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2506.09501.
- [9] H. He and Thinking Machines Lab, "Defeating nondeterminism in LLM inference," Thinking Machines Lab: Connectionism, technical report, 2025, accessed 15 August 2026, doi:10.64434/tml.20250910.
- [10] W. Hoeffding, "Probability inequalities for sums of bounded random variables," *J. Amer. Statist. Assoc.*, vol. 58, no. 301, pp. 13–30, 1963, doi:10.1080/01621459.1963.10500830.
- [11] D. Kang, E. Gan, P. Bailis, T. Hashimoto, and M. Zaharia, "Approximate selection with guarantees using proxies," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 1990–2003, 2020, doi:10.14778/3407790.3407804.
- [12] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934, doi:10.1093/biomet/26.4.404.
- [13] A. Maurer and M. Pontil, "Empirical Bernstein bounds and sample variance penalization," in *Proc. Conf. on Learning Theory (COLT)*, 2009, online proceedings, paper 012; also arXiv:0907.3740.
- [14] R. Bardenet and O.-A. Maillard, "Concentration inequalities for sampling without replacement," *Bernoulli*, vol. 21, no. 3, pp. 1361–1385, 2015, doi:10.3150/14-BEJ605.
- [15] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, "Learn then test: Calibrating predictive algorithms to achieve risk control," *Ann. Appl. Stat.*, vol. 19, no. 2, pp. 1641–1662, 2025, arXiv:2110.01052, doi:10.1214/24-AOAS1998.
- [16] F. Gilardi, M. Alizadeh, and M. Kubli, "ChatGPT outperforms crowd workers for text-annotation tasks," *Proc. Natl. Acad. Sci. USA*, vol. 120, no. 30, p. e2305016120, 2023, doi:10.1073/pnas.2305016120.
- [17] C. Ziems, W. Held, O. Shaikh, J. Chen, Z. Zhang, and D. Yang, "Can large language models transform computational social science?" *Comput. Linguist.*, vol. 50, no. 1, pp. 237–291, 2024, doi:10.1162/coli_a_00502.
- [18] N. Pangakis, S. Wolken, and N. Fasching, "Automated annotation with generative AI requires validation," arXiv preprint arXiv:2306.00176, 2023.
- [19] P. Törnberg, "ChatGPT-4 outperforms experts and crowd workers in annotating political Twitter messages with zero-shot learning," arXiv preprint arXiv:2304.06588, 2023.
- [20] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proc. 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, 2023, pp. 212–218, doi:10.18653/v1/2023.nlposs-1.24.
- [21] R. J. Serfling, "Probability inequalities for the sum in sampling without replacement," *Ann. Statist.*, vol. 2, no. 1, pp. 39–48, 1974, doi:10.1214/aos/1176342611.
- [22] I. Waudby-Smith and A. Ramdas, "Estimating means of bounded random variables by betting," *J. R. Stat. Soc. Series B*, vol. 86, no. 1, pp. 1–27, 2024, doi:10.1093/jrsssb/qqad009.
- [23] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023, arXiv:2203.11171.
- [24] J. Lao, A. Zimmerer, O. Ovcharenko, T. Cong, M. Russo, G. Vitagliano, M. Cochez, F. Özcan, G. Gupta, T. Hottelier, H. V. Jagadish, K. Kissel, S. Schelter, A. Kipf, and I. Trummer, "SemBench: A benchmark for semantic query processing engines," *Proc. VLDB Endow.*, vol. 19, no. 8, pp. 1754–1767, 2026, doi:10.14778/3811243.3811249.
- [25] L. Patel, S. Jha, M. Pan, H. Gupta, P. Asawa, C. Guestrin, and M. Zaharia, "Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4171–4184, 2025, doi:10.14778/3749646.3749685.
- [26] D. Kang, J. Guibas, P. Bailis, T. Hashimoto, Y. Sun, and M. Zaharia, "Accelerating approximate aggregation queries with expensive predicates," *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 2341–2354, 2021, doi:10.14778/3476249.3476285.
- [27] S. Zeighami, S. Shankar, and A. Parameswaran, "Cut costs, not accuracy: LLM-powered data processing with guarantees," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–26, 2025, SIGMOD 2026, doi:10.1145/3769776.
- [28] S. Jo and I. Trummer, "ThalamusDB: Approximate query processing on multi-modal data," *Proc. ACM Manag. Data*, vol. 2, no. 3, pp. 1–26, 2024, doi:10.1145/3654989.
- [29] S. Shankar, S. Zeighami, and A. Parameswaran, "Task cascades for efficient unstructured data processing," *Proc. ACM Manag. Data*, vol. 4, no. 1, pp. 1–26, 2026, SIGMOD 2026, doi:10.1145/3786702.
- [30] S. R. Howard, A. Ramdas, J. McAuliffe, and J. Sekhon, "Time-uniform, nonparametric, nonasymptotic confidence sequences," *Ann. Statist.*, vol. 49, no. 2, pp. 1055–1080, 2021, doi:10.1214/20-AOS1991.
- [31] S. Bates, A. Angelopoulos, L. Lei, J. Malik, and M. I. Jordan, "Distribution-free, risk-controlling prediction sets," *J. ACM*, vol. 68, no. 6, pp. 43:1–43:34, 2021, doi:10.1145/3478535.
- [32] C. Liu, M. Russo, M. Cafarella, L. Cao, P. B. Chen, Z. Chen, M. Franklin, T. Kraska, S. Madden, R. Shahout, and G. Vitagliano, "Palimpsest: Optimizing AI-powered analytics with declarative query processing," in *Proc. Conf. on Innovative Data Systems Research (CIDR)*, 2025, <https://vldb.org/cidrdb/papers/2025/p12-liu.pdf>, accessed 30 September 2026.
- [33] S. Shankar, T. Chambers, T. Shah, A. G. Parameswaran, and E. Wu, "DocETL: Agentic query rewriting and evaluation for complex document

- processing,” *Proc. VLDB Endow.*, vol. 18, no. 9, pp. 3035–3048, 2025, doi:10.14778/3746405.3746426.
- [34] S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, “Language models enable simple systems for generating structured views of heterogeneous data lakes,” *Proc. VLDB Endow.*, vol. 17, no. 2, pp. 92–105, 2023, doi:10.14778/3626292.3626294.
- [35] H. Dai, B. Y. Wang, X. Wan, B. Dai, S. Yang, A. Nova, P. Yin, P. M. Phothilimthana, C. Sutton, and D. Schuurmans, “UQE: A query engine for unstructured databases,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2407.09522.
- [36] H. F. Dodge and H. G. Romig, “A method of sampling inspection,” *Bell Syst. Tech. J.*, vol. 8, no. 4, pp. 613–631, 1929, doi:10.1002/j.1538-7305.1929.tb01240.x.
- [37] S. J. Pocock, “Group sequential methods in the design and analysis of clinical trials,” *Biometrika*, vol. 64, no. 2, pp. 191–199, 1977, doi:10.1093/biomet/64.2.191.
- [38] K. K. G. Lan and D. L. DeMets, “Discrete sequential boundaries for clinical trials,” *Biometrika*, vol. 70, no. 3, pp. 659–663, 1983, doi:10.1093/biomet/70.3.659.
- [39] M. Lindeman and P. B. Stark, “A gentle introduction to risk-limiting audits,” *IEEE Secur. Privacy*, vol. 10, no. 5, pp. 42–49, 2012, doi:10.1109/MSP.2012.56.
- [40] L. G. Schroeder, A. Desai, A. Cuadron, K. Chu, S. Liu, M. Zhao, S. Krusche, A. Kemper, M. Zaharia, and J. E. Gonzalez, “vCache: Verified semantic prompt caching,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2026, arXiv:2502.03771.
- [41] A. W. Lee, B. Han, S. Sen, S. Yeom, U. Çetintemel, and A. Datta, “Evergreen: Efficient claim verification for semantic aggregates,” arXiv preprint arXiv:2604.26180, 2026.
- [42] M. Mansoor, T. Ahmad, and Y.-C. Yoon, “Risk-constrained freshness-aware semantic caching for open-web retrieval-augmented LLMs,” arXiv preprint arXiv:2607.04281, 2026.
- [43] S. Krishnan, J. Wang, M. J. Franklin, K. Goldberg, and T. Kraska, “Stale view cleaning: Getting fresh answers from stale materialized views,” *Proc. VLDB Endow.*, vol. 8, no. 12, pp. 1370–1381, 2015, doi:10.14778/2824032.2824037.
- [44] J. M. Hellerstein, P. J. Haas, and H. J. Wang, “Online aggregation,” in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 1997, pp. 171–182, doi:10.1145/253260.253291.
- [45] M. Budiu, T. Chajed, F. McSherry, L. Ryzhyk, and V. Tannen, “DBSP: Automatic incremental view maintenance for rich query languages,” *Proc. VLDB Endow.*, vol. 16, no. 7, pp. 1601–1614, 2023, doi:10.14778/3587136.3587137.
- [46] A. Gupta, I. S. Mumick, and K. A. Ross, “Adapting materialized views after redefinitions,” in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 1995, pp. 211–222, doi:10.1145/223784.223817.
- [47] S. Chen, J. Liu, D. Raghavan, and U. Çetintemel, “VectraFlow: Long-horizon semantic processing over data and event streams with LLMs,” arXiv preprint arXiv:2604.03855, 2026.
- [48] S. Chen, D. Raghavan, and U. Çetintemel, “Continuous prompts: LLM-augmented pipeline processing over unstructured streams,” arXiv preprint arXiv:2512.03389, 2025.
- [49] P. Liskowski and K. Schmaus, “Streaming model cascades for semantic SQL,” arXiv preprint arXiv:2604.00660, 2026.
- [50] A. Nouri, “StepCache: Step-level reuse with lightweight verification and selective patching for LLM serving,” arXiv preprint arXiv:2603.28795, 2026.
- [51] D. M. L. Martins and G. Vossen, “Prompt as a data type: In-database LLM prompt management and rewriting,” arXiv preprint arXiv:2607.21756, 2026.
- [52] U. Çetintemel, S. Chen, A. W. Lee, D. Raghavan, D. Lu, and A. Crotty, “Making prompts first-class citizens for adaptive LLM pipelines,” in *Proc. Conf. on Innovative Data Systems Research (CIDR)*, 2026, arXiv:2508.05012.
- [53] S. Shankar, H. Li, P. Asawa, M. Hulsebos, Y. Lin, J. D. Zamfirescu-Pereira, H. Chase, W. Fu-Hinthorn, A. G. Parameswaran, and E. Wu, “SPADE: Synthesizing data quality assertions for large language model pipelines,” *Proc. VLDB Endow.*, vol. 17, no. 12, pp. 4173–4186, 2024, doi:10.14778/3685800.3685835.
- [54] T. Dixit, D. Lee, S. Fang, S. S. Harsha, A. Sureshan, A. Maharaj, and Y. Li, “RETAIN: Interactive tool for regression testing guided LLM migration,” in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2024, pp. 301–310, arXiv:2409.03928, doi:10.18653/v1/2024.emnlp-demo.31.
- [55] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, “Quantifying language models’ sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024, arXiv:2310.11324.
- [56] L. Chen, M. Zaharia, and J. Zou, “How is ChatGPT’s behavior changing over time?” *Harvard Data Sci. Rev.*, vol. 6, no. 2, 2024, doi:10.1162/99608f92.5317da47.
- [57] R. Wang and A. Ramdas, “False discovery rate control with e-values,” *J. R. Stat. Soc. Series B*, vol. 84, no. 3, pp. 822–852, 2022, doi:10.1111/rssb.12489.
- [58] Stack Overflow, “Stack Overflow annual developer survey 2023,” 2023, accessed 1 October 2026. [Online]. Available: <https://survey.stackoverflow.co/2023/>
- [59] N. Drushchak and M. Romanysyn, “Introducing the Djinni recruitment dataset: A corpus of anonymized CVs and job postings,” in *Proc. 3rd Ukrainian Natural Language Processing Workshop (UNLP) at LREC-COLING 2024*, 2024, pp. 8–13, doi:10.63317/2dcvy45ws6yb.

ARJUN LOHAN received the B.S. degree in chemical engineering from the University of California San Diego, La Jolla, CA, USA, in 2019, and the M.S. degree in financial engineering from the University of Southern California, Los Angeles, CA, USA, in 2024, and is currently affiliated with the University of Southern California. Research interests include data systems for language-model pipelines, semantic caching and materialized-view maintenance, and statistical guarantees for machine-computed data.

...